

iOS Upload Guide & Version Number Gotchas

What this covers

A focused reference for the upload-to-Apple part of the pipeline, plus everything about version numbers, running the simulator for quick testing, and common build errors. Pair this with the full `iOS Release Runbook` for the complete end-to-end process.

Your setup: code written on Windows / VS Code, pushed to GitHub, then built and uploaded from the Proxmox macOS VM (HACK-SONOMA) over SSH. No local Xcode GUI session involved in the upload path.

0. Push Your Code Changes First

Windows · VS Code terminal

Before doing anything on the Mac VM, make sure your latest code is actually on GitHub — the Mac VM builds whatever `main` contains, nothing more.

```
git add -A
git commit -m "describe what changed"
git push origin main
```

If asked for a password, paste your GitHub **Personal Access Token** (starts with `ghp_`), not your account password.

1. Pull the Latest Code and Build

Proxmox Mac VM · SSH

Make sure this machine has whatever you last pushed from VS Code — otherwise you'll sign and upload stale code.

```
cd ~/Documents/quran-app
git checkout main
git pull origin main
flutter pub get
```

If git pull refuses because of local changes

The Mac VM sometimes carries its own small local edits (e.g. the Arabic display name in `Info.plist`). Don't discard them blindly — stash, pull, then reapply:

```
git stash
git pull origin main
git stash pop
```

If `git stash pop` reports a conflict, open the file, keep **both** sets of changes, then `git add <file>` and commit.

Then unlock the keychain (needed every SSH session before signing) and build:

```
security unlock-keychain -p "<YOUR_MAC_PASSWORD>" ~/Library/Keychains/login.keychain-db
security set-key-partition-list -S apple-tool:,apple: -s -k "<YOUR_MAC_PASSWORD>" ~/Library/Keychains/login.keychain-db

flutter build ipa
```

Keep the password out of saved files

The two `security` commands above include your Mac login password in plaintext on the command line. That's necessary for unattended SSH signing — but never paste the real password into a document you commit to GitHub or share. Type it fresh each time, or keep it only in a password manager.

Success ends with `✓ Built IPA to build/ios/ipa`. The launch-image warning is cosmetic and safe to ignore for TestFlight.

2. The Upload Command

Proxmox Mac VM · SSH

Once the IPA is built, upload it:

```
cd ~/Documents/quran-app

xcrun altool --upload-app --type ios \
  -f build/ios/ipa/*.ipa \
  --apiKey 929S7G7VV7 \
  --apiIssuer 1146c4bd-fbd8-426d-b08e-6dd221af17b3
```

Success looks like:

```
UPLOAD SUCCEEDED with no errors
Delivery UUID: <a long UUID>
```

"WILL RETRY PART n" / "The network connection was lost" — not a failure

During upload you will often see a wall of red `ERROR` lines like this:

```
ERROR: [ContentDelivery.Uploader...] WILL RETRY PART 2. Failed with error:
Error Domain=NSURLErrorDomain Code=-1005 "The network connection was lost."
```

This looks alarming but is usually harmless. `altool` uploads the IPA as many parallel chunks and automatically retries any that drop mid-transfer. As long as the run ends with `UPLOAD SUCCEEDED with no errors` and a Delivery UUID, the upload completed — ignore the retry noise above it.

Why it happens on this setup: flaky connection between the Proxmox VM's virtual NIC and Apple's upload servers during a large (~123MB) multi-part transfer. This VM has had this before, traced to the MTU setting.

If retries happen a lot, or an upload never recovers:

- Confirm the VM's MTU is 1500 with TSO disabled:

```
ifconfig en0 | grep mtu
# should show mtu 1500. If not:
sudo ifconfig en0 mtu 1500
```

- Re-run the exact same `xcrun altool` command — safe to retry if the first attempt never printed `UPLOAD SUCCEEDED`.
- If it fails repeatedly (not just noisy retries), check Proxmox host network load and the VM's virtual NIC driver — not the app or Flutter project.

3. Version Number Gotchas

The two numbers in pubspec.yaml

The line in `pubspec.yaml` looks like `version: 1.1.3+19` (your current value). It contains two separate values Apple tracks independently:

Part	Apple's name	Rule
1.1.3	CFBundleShortVersionString (“Version”)	What users see on the App Store. Must be higher than the last approved / released version.
19	CFBundleVersion (“Build”)	Must be higher than the last uploaded build — even if that upload was for the same version and was never released.

The rule of thumb — which number to bump

- Version already **live on the App Store** → bump the **version** (e.g. `1.1.3` → `1.1.4`) **and** the **build** (e.g. `+19` → `+20`). Both.
- Re-uploading because the last upload **failed or was rejected**, and the version was never approved → bump **only the build** number (e.g. `+19` → `+20`). Leave the version alone.
- Starting a new version you've been testing and burning builds on → bump **both**: the version goes up, and the build goes up past whatever test builds you already used.

When in doubt: always bump the build number. You can never go wrong bumping it. You can get an error if it's too low.

Errors this causes, and exactly what they mean

Error text from Apple	Meaning & Fix
CFBundleShortVersionString [1.1.3] ... must contain a higher version than that of the previously approved version [1.1.3]	That version is already live. Bump the version number, not just the build. Rebuild and re-upload.
Invalid Pre-Release Train. The train version '1.1.3' is closed for new build submissions	Same root cause — version window closed. Bumping the version fixes both errors together.
“Redundant binary upload” / build already used	Same build number uploaded twice. Bump only the build number and re-upload.

Editing the version

Either machine

Open `pubspec.yaml` and edit the `version:` line by hand. Or use `sed` from the terminal — but **always substitute the actual current values** from your file, not a copied example:

```
# First, check what the current value actually is:
grep '^version:' pubspec.yaml

# Then edit it. Example only – replace OLD and NEW with real values:
sed -i '' 's/version: OLD_VERSION+OLD_BUILD/version: NEW_VERSION+NEW_BUILD/' pubspec.yaml
# e.g.: sed -i '' 's/version: 1.1.3+19/version: 1.1.4+20/' pubspec.yaml

# On Windows PowerShell / VS Code, just edit the line by hand and save.
```

sed silently does nothing if the pattern doesn't match

If you copy a `sed` example with stale version numbers and the pattern doesn't exactly match what's in your file, `sed` produces no error and makes no change. Always run `grep '^version:' pubspec.yaml` before and after to confirm the edit actually happened.

4. How to Find Out the Current Live Version

Three independent places to check — use more than one if ever unsure what is live.

A. In your code (before you upload)

VS Code or terminal, either machine

```
grep '^version:' pubspec.yaml
```

This tells you what your *next* upload will carry. It does **not** tell you what is live — only what you're about to send.

B. App Store Connect website (source of truth)

Browser

1. Go to `appstoreconnect.apple.com` → **My Apps** → your app.
2. The **App Store** tab shows the version currently **live or in review** at the top with a status badge: *Ready for Sale*, *In Review*, *Pending Developer Release*, etc.
3. **App Store** → **Version History** (left sidebar) shows every version ever released with dates. This is the definitive record.
4. The **TestFlight** tab lists every *build* number ever uploaded. Use this to find the highest build number you've used so far — your next build must be higher.

Quick pre-upload check

App Store Connect → your app → **App Store** tab: whatever version shows as “Ready for Sale” is your version floor — your next version must be higher. Then **TestFlight** tab: highest build number listed is your build floor — your next build must be higher than that, regardless of which version it belonged to.

5. Running on the Simulator (Quick Testing, No Upload Needed)

Use this when you want to quickly test a change on a simulated device without going through the full build-ipa → upload → TestFlight cycle. This is the right path for day-to-day iteration.

From SSH (no GUI needed — the normal workflow for this VM)

Proxmox Mac VM - SSH

```
cd ~/Documents/quran-app
git pull origin main
flutter pub get
flutter devices          # confirm a simulator is listed and booted
```

Then run on the simulator:

```
flutter run
```

Flutter picks the first available simulator automatically. To target a specific one:

```
# List available simulators
xcrun simctl list devices | grep -E "Booted|Shutdown" | head -20

# Boot one if none are running
xcrun simctl boot "iPhone 16"

# Run on it
flutter run -d "iPhone 16"
```

If Xcode says "Module not found" when you try to open the workspace

This happens when CocoaPods dependencies (like `flutter_local_notifications`) are missing because the `Podfile` and `Pods/` folder are not committed to git (they're gitignored). The fix is to regenerate them on the Mac:

```
cd ~/Documents/quran-app
flutter clean
flutter pub get
flutter build ios --config-only # regenerates Podfile and runs pod install
```

After that, always open the **workspace** (`.xcworkspace`), not the project (`.xcodeproj`) — Xcode only knows about the pods through the workspace file:

```
open ios/Runner.xcworkspace
```

If Xcode shows a dialog asking whether to “keep the Xcode version or use the version on disk” after you run the above commands — choose **"Use the version on disk."** That's the freshly regenerated file with pods wired in.

CocoaPods must be installed: `pod --version` should return a version number. If missing: `brew install cocoapods`.

Simulator vs. TestFlight — which to use when

- **Simulator + `flutter run`** — fastest, no signing, no upload. Good for UI changes, logic fixes, layout tweaks. Background audio and some hardware-dependent features don't work reliably in the simulator.
- **TestFlight on a real iPhone** — required for anything involving audio, notifications, camera, or any feature that behaves differently on real hardware. Always test on device before submitting to the App Store.

6. After a Successful Upload

1. Wait 5–15 minutes for Apple to finish *Processing* the build (visible in **TestFlight** tab → **Builds**).
2. Clear the **Export Compliance** prompt if it appears: no custom encryption — standard HTTPS only.
3. Install via the **TestFlight** app on a real iPhone and test the actual change.
4. Only once verified working: open the version page under the **App Store** tab, attach this build, fill in “What’s New,” and **Submit for Review**.
5. Push the version bump to GitHub so all machines stay in sync:

```
git add pubspec.yaml
git commit -m "bump version for release"
git push origin main
```

Reference values: Bundle ID `com.mahfodqr.islamicDawahMushaf` · Team ID `Q2W89UQXH9` · API Key ID `929S7G7VV7` · Issuer ID `1146c4bd-fbd8-426d-b08e-6dd221af17b3` · Current version as of last release: `1.1.3+19`